

SOLID principles and hexagonal architecture with FastAPI and use of relational DB

Activity data

Activity number	3
Professor	Ramon Amela Milian
Teaching language	English
Association	Individual
Delivery date	06-07-2026

Activity description

In this activity, HTTP methods, SOLID principles, hexagonal architecture, and relational databases will be worked on. The student will learn how to build an HTTP API using FastAPI. The starting point will be a minimal running FastAPI project, and some exercises will be done to achieve a simple but complete, clean, and scalable API. More precisely, we will develop the API for a storage file system.

On the one hand, an authentication API will be developed to register and authenticate users. At this point, an external identifier should be generated for each user. This external identifier should be an integer (and be unique for each one of the users). This external identifier should be stored next to files to identify the user to whom the file belongs to. All the information related to the user and relations with the files will be stored in a relational database following hexagonal architecture and dependency injection.

It is recommended to start developing from the result obtained in the second deliverable so that the result is better. Nevertheless, it is possible to start with the available template, and all the missing pieces from the second deliverable will not be considered nor penalized. More precisely, what will be evaluated in this deliverable is the endpoints implementation, the use of tortoise as ORM, PostgreSQL as relational databases in a FastAPI project and the global organization of it with SOLID principles and hexagonal architecture.

Competences

CE3 – Design and manage information systems for data treatment and storage

COMPETENCES (CE / CT)	LEARNING RESULTS (RA)	EVALUATION (IE)	INDICATORS
CE3	RA3 – Justify the use of cloud technologies to guarantee the scalability, accessibility, and coherence of a software solution	IE4 – The student can implement modern architecture to guarantee the scalability and maintainability of the project	
	RA4 – Value the correct interaction between different cloud technologies during the design of a data processing solution	IE5 – The student can design an API that interacts with a relational database	

Activity contents

- HTTP project organization
 - o SOLID principles
 - o Hexagonal architecture
- Use of relational databases
 - o PostgreSQL
- Use of ORM
 - o Tortoise
- Use of HTTP frameworks
 - o FastAPI

Task descriptions

- Create two apps called “authentication” and “files”
- Create routers inside both new applications
- Create files named “models.py” inside each folder with the tortoise definitions to be stored in the relational database
- Import the new routers from the main file
- Create four folders inside each one of the apps:
 - api
 - dependency_injection
 - domain
 - persistence
- Create the following endpoints in the newly created folders.

In the following definitions, it is possible to identify the user with a string or with an email. Choose what you think is the best option.

It is important to note that always that the session token is passed as input, it should be sent through a header called “Auth”.

- **Authentication**
 - register – POST

Input: at least, the minimum data that you consider necessary to code all the other endpoints.

Action: create a new user unto the database

Output: code errors for the considered cases (for example, 200 for success, 409 when user already exists).

- login – POST

Input: user id and password.

Action: create a new session for this user

Output: connection token and code for success/errors

- logout – POST

Input: connection token

Action: close the session associated to the input token

Output: code for success/errors

- introspect – GET

Input: session token

Action: no action

Output: verify whether the token is correct or not and, in case it is, return the information associated to the user as well as the code for success/errors

- **Files**

- files – GET

Input: session token

Action: no effect

Output: return the information for all the files owned by the user corresponding to the token passed in the call and the code for success/errors

- files – POST

Input: file information and session token

Action: create a new file

Output: return the id for the created file and the code for success/errors

- files/{id} – GET

Input: file id and session token

Action: no action

Output: return the information (and content if present) for the id given in the input and the code for success/errors

- files/{id} – DELETE

Input: file id and session token

Action: delete the file with the id passed as parameter

Output: code for success/errors

- files/{id} – POST

Input: file id, session token and file content

Action: fill in the file content

Output: code for success/errors

- files/merge – POST

Input: several file ids, session token

Action: merge pdf's

Output: the id of the merged file and code for success/errors

It is important to note that there are two different endpoints to create files. One to upload the file information (like filename, description... and whatever information wants to be saved) and one to upload the file content.

The API should contain the routers. The domain should have the code containing the endpoint logics (token verifications, consistency checks...) and the business objects. Persistence should contain the class implementing all persistence operations to interact with the Postgres database. Finally, singletons linking the API with the domain in will be placed in dependency_injection.

In addition:

- The root of each app (authentication and files) should contain a file named “models” with the persistence definitions for the ORM.
- You should initialize the database from the previous models
- Add one target to the docker-compose file named “make_migrations” from the service image and the following endpoint:
 - o entrypoint: aerich migrate
- Add one target to the docker-compose file named “migrate” from the service image and the following endpoint:
 - o entrypoint: aerich upgrade

All the previous steps could be separated in the following summary points:

1. Use of dependency injection
2. Use of business objects to represent the domain objects
3. Use of SOLID principles to code the domain
4. Use of relational databases (PostgreSQL) and ORM (*tortoise*)

Formal aspects of tasks

All the project’s code must be delivered in compressed format through the virtual campus. In addition, in the delivered text, the commit hash with the code in *GitHub* should be included. This hash should contain the same exact version as the delivered code.

Task	Percentage
1	15%
2	15%
3	35%
4	35%

Evaluation

Number	Task	Weight	IE
3	SOLID principles and hexagonal architecture with FastAPI and use of relational DB	25%	IE4, IE5

Information resources

Capsules and videoconferences